

USER_GUIDE

User Guide

Introduction

DocProcessor loads project documentation, extracts features, and tracks verification coverage. This guide walks through common usage patterns.

Installation

```
go install digital.vasic.docprocessor/cmd/docprocessor@latest
```

Or build from source:

```
git clone git@github.com:vasic-digital/DocProcessor.git
cd DocProcessor
go build ./cmd/docprocessor
```

Tutorial

Step 1: Load Documents

```
package main

import (
    "context"
    "fmt"
    "digital.vasic.docprocessor/pkg/loader"
)

func main() {
    l := loader.NewDefaultLoader([]string{"md", "yaml", "html"})
    docs, err := l.LoadDir(context.Background(), "./docs")
    if err != nil {
        panic(err)
    }
}
```

```

    fmt.Printf("Loaded %d documents\n", len(docs))
}

```

Step 2: Build Feature Map

```

builder := feature.NewBuilder("/path/to/project")
fm, err := builder.BuildFromDocs(ctx, docs)
// fm.Features, fm.Categories, fm.PlatformMatrix now populated

```

Step 3: Enrich with LLM (Optional)

```

// Implement the llm.LLMAgent interface
agent := myLLMAgent{}
err := builder.Enrich(ctx, fm, agent)
// fm.Screens and fm.Features[i].TestSteps now populated

```

Step 4: Track Coverage

```

tracker := coverage.NewTracker()

for _, f := range fm.Features {
    tracker.RegisterFeature(
        coverage.Feature{ID: f.ID, Name: f.Name, Category:
            string(f.Category)},
        []string{"android", "desktop", "web"},
    )
}

// During verification...
tracker.MarkVerified("feat-markdown", "android",
    coverage.Evidence{
        ScreenshotPath: "/evidence/markdown-android.png",
    })

report := tracker.Coverage()
fmt.Printf("Coverage: %.1f%%\n", report.OverallPct*100)

```

Step 5: Export Results

```

// Export doc graph
jsonData, _ := fm.DocGraph.ExportJSON()
mermaid := fm.DocGraph.ExportMermaid()

// Export coverage
snapshot := tracker.Export()

```

Configuration

Create a `.env` file from `.env.example`:

```
cp .env.example .env
```

Key settings:

- `HELIX_DOCS_ROOT` -- Root directory for documentation
- `HELIX_DOCS_AUTO_DISCOVER` -- Auto-discover docs by patterns
- `HELIX_DOCS_FORMATS` -- Supported file extensions

Supported Formats

Format	Extensions	Parser
Markdown	.md	Section/link extraction
YAML	.yaml, .yml	Key-value section extraction
HTML	.html	Raw content with title
AsciiDoc	.adoc	Raw content with title
reStructuredText	.rst	Raw content with title

Concurrency

CoverageTracker is thread-safe. Multiple goroutines can call `MarkVerified`, `MarkFailed`, and `MarkSkipped` concurrently. Read operations (`Coverage()`, `Unverified()`) can run in parallel with each other.

DocGraph is also thread-safe with `sync.RWMutex` protection on all operations.